

PROJECT DESCRIPTION

In this lab activity, I performed tasks associated with using tcpdump to capture network traffic. I captured the data in a packet capture (p-cap) file and then examined the contents of the captured packet data to focus on specific types of traffic.

SCENARIO

I am a network analyst who needs to use tcpdump to capture and analyze live network traffic from a Linux virtual machine.

FIRST STEP: IDENTIFY NETWORK INTERFACES

In this task, I identified network interfaces that can be used to capture network packet data.

- I used **ifconfig** to identify the interfaces that are available, this command returns the following output :

```
analyst@943bf9bca394:~$ sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1460
    inet 172.17.0.2 netmask 255.255.0.0 broadcast 172.17.255
.255
    ether 02:42:ac:11:00:02 txqueuelen 0 (Ethernet)
    RX packets 842 bytes 14002103 (13.3 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 488 bytes 43045 (42.0 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 110 bytes 13748 (13.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 110 bytes 13748 (13.4 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

- I used **tcpdump** to identify the interface options available for packet capture, this command also allows me to identify which network interfaces are available. This may be useful on systems that do not include the **ifconfig** command:

```
analyst@943bf9bca394:~$ sudo tcpdump -D
1.eth0 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
5.nflog (Linux netfilter log (NFLOG) interface) [none]
6.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
7.dbus-system (D-Bus system bus) [none]
8.dbus-session (D-Bus session bus) [none]
```

SECOND STEP: INSPECT THE NETWORK TRAFFIC OF A NETWORK INTERFACE

WITH tcpdump

In this task, I used **tcpdump** to filter live network packet traffic on an interface.

- Filter live network packet data from the **eth0** interface with **tcpdump**: **sudo tcpdump -i eth0 -v -c5**. This command will run **tcpdump** with the following options:
 - **-i eth0**: Capture data specifically from the **eth0** interface.
 - **-v**: Display detailed packet data.
 - **-c5**: Capture 5 packets of data.

```
analyst@943bf9bca394:~$ sudo tcpdump -i eth0 -v -c5
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot
length 262144 bytes
08:29:00.011837 IP (tos 0x0, ttl 64, id 2931, offset 0, flags [DF]
, proto TCP (6), length 121)
    943bf9bca394.5000 > nginx-us-west1-c.c.qwiklabs-terminal-vms-p
rod-00.internal.54698: Flags [P.], cksum 0x591e (incorrect -> 0xbf
74), seq 3689903553:3689903622, ack 1425808922, win 998, options [
nop,nop,TS val 167884883 ecr 3400867066], length 69
08:29:00.013090 IP (tos 0x0, ttl 63, id 18055, offset 0, flags [DF
], proto TCP (6), length 52)
    nginx-us-west1-c.c.qwiklabs-terminal-vms-prod-00.internal.5469
8 > 943bf9bca394.5000: Flags [.], cksum 0xc38d (correct), ack 69,
win 507, options [nop,nop,TS val 3400867128 ecr 167884883], length
0
08:29:00.080161 IP (tos 0x0, ttl 64, id 3605, offset 0, flags [DF]
, proto UDP (17), length 71)
    943bf9bca394.42660 > metadata.google.internal.domain: 61915+ P
TR? 137.0.22.172.in-addr.arpa. (43)
08:29:00.084346 IP (tos 0x0, ttl 64, id 2932, offset 0, flags [DF]
, proto TCP (6), length 142)
    943bf9bca394.5000 > nginx-us-west1-c.c.qwiklabs-terminal-vms-p
rod-00.internal.54698: Flags [P.], cksum 0x5933 (incorrect -> 0xb0
8a), seq 69:159, ack 1, win 998, options [nop,nop,TS val 167884955
ecr 3400867128], length 90
```

THIRD STEP: CAPTURE NETWORK TRAFFIC WITH **tcpdump**

In this task, I used **tcpdump** to save the captured network data to a packet capture file.

In the previous command, I used **tcpdump** to stream all network traffic. Here, I used a filter and other **tcpdump** configuration options to save a small sample that contains only web (TCP port 80) network packet data.

1. I Captured packet data into a file called `capture.pcap`:`sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap &`

This command will run `tcpdump` in the background with the following options:

- **-i eth0**: Capture data from the `eth0` interface.
- **-nn**: Do not attempt to resolve IP addresses or ports to names.
- **-c9**: Capture 9 packets of data and then exit.
- **port 80**: Filter only port 80 traffic. This is the default HTTP port.
- **-w capture.pcap**: Save the captured data to the named file.
- **&**: This is an instruction to the Bash shell to run the command in the background.

```
analyst@943bf9bca394:~$ sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap &
[1] 13563
analyst@943bf9bca394:~$ tcpdump: listening on eth0, link-type EN10
MB (Ethernet), snapshot length 262144 bytes
```

2. I Used `curl` to generate some HTTP (port 80) traffic & verified the packet data capture.

```
analyst@943bf9bca394:~$ curl opensource.google.com
<HTML><HEAD><meta http-equiv="content-type" content="text/html;char
rset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="https://opensource.google/">here</A>.
</BODY></HTML>
analyst@943bf9bca394:~$ 9 packets captured
10 packets received by filter
0 packets dropped by kernel
```

```
analyst@943bf9bca394:~$ ls -l capture.pcap
-rw-r--r-- 1 tcpdump tcpdump 1401 Dec 21 08:45 capture.pcap
```

FINAL STEP: FILTER THE CAPTURED PACKET DATA

In this task, I used **tcpdump** to filter data from the packet capture file I saved previously.

1. I Used the **tcpdump** command to filter the packet header data from the **capture.pcap** capture file: **sudo tcpdump -nn -r capture.pcap -v**

This command will run **tcpdump** with the following options:

- **-nn**: Disable port and protocol name lookup.
- **-r**: Read capture data from the named file.
- **-v**: Display detailed packet data.

```
analyst@943bf9bca394:~$ sudo tcpdump -nn -r capture.pcap -v
reading from file capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
08:45:17.369008 IP (tos 0x0, ttl 64, id 24477, offset 0, flags [DF], proto TCP (6), length 60)
    172.17.0.2.33048 > 108.177.98.100.80: Flags [S], cksum 0x7b57 (incorrect -> 0xe810), seq 1238394386, win 65320, options [mss 1420,sackOK,TS val 3080060133 ecr 0,nop,wscale 6], length 0
08:45:17.369814 IP (tos 0x0, ttl 126, id 0, offset 0, flags [DF], proto TCP (6), length 60)
    108.177.98.100.80 > 172.17.0.2.33048: Flags [S.], cksum 0xb3d5 (correct), seq 3867332386, ack 1238394387, win 65535, options [mss 1420,sackOK,TS val 2621497706 ecr 3080060133,nop,wscale 8], length 0
08:45:17.369842 IP (tos 0x0, ttl 64, id 24478, offset 0, flags [DF], proto TCP (6), length 52)
    172.17.0.2.33048 > 108.177.98.100.80: Flags [.], cksum 0x7b4f (incorrect -> 0xde7c), ack 1, win 1021, options [nop,nop,TS val 3080060134 ecr 2621497706], length 0
08:45:17.369938 IP (tos 0x0, ttl 64, id 24479, offset 0, flags [DF], proto TCP (6), length 137)
    172.17.0.2.33048 > 108.177.98.100.80: Flags [P.], cksum 0x7ba4 (incorrect -> 0x4c30), seq 1:86, ack 1, win 1021, options [nop,nop,TS val 3080060134 ecr 2621497706], length 85: HTTP, length: 85
    GET / HTTP/1.1
    Host: opensource.google.com
    User-Agent: curl/7.74.0
    Accept: */*
08:45:17.370189 IP (tos 0x0, ttl 126, id 0, offset 0, flags [DF], proto TCP (6), length 52)
    108.177.98.100.80 > 172.17.0.2.33048: Flags [.], cksum 0x0e09 (correct), ack 86, win 1051, options [nop,nop,TS val 2621497706 ecr 3080060134], length 0
08:45:17.372095 IP (tos 0x0, ttl 126, id 0, offset 0, flags [DF], proto TCP (6), length 590)
    108.177.98.100.80 > 172.17.0.2.33048: Flags [P.], cksum 0xab19 (correct), seq 1:539, ack 86, win 1051, options [nop,nop,TS val 2621497708 ecr 3080060134], length 538: HTTP, length: 538
    HTTP/1.1 301 Moved Permanently
    Location: https://opensource.google/
    Content-Type: text/html; charset=UTF-8
    X-Content-Type-Options: nosniff
    Date: Sun, 21 Dec 2025 08:45:17 GMT
```

I must specify the **-nn** switch again here, as I want to make sure **tcpdump** does not perform name lookups of either IP addresses or ports, since this can alert threat actors.

2. I Used the **tcpdump** command to filter the extended packet data from the **capture.pcap** capture file: **sudo tcpdump -nn -r capture.pcap -X**. This command will run **tcpdump** with the following options:
- **-nn**: Disable port and protocol name lookup.
 - **-r**: Read capture data from the named file.
 - **-X**: Display the hexadecimal and ASCII output format packet data.

```

analyst@943bf9bca394:~$ sudo tcpdump -nn -r capture.pcap -X
reading from file capture.pcap, link-type EN10MB (Ethernet), snaps
hot length 262144
08:45:17.369008 IP 172.17.0.2.33048 > 108.177.98.100.80: Flags [S]
, seq 1238394386, win 65320, options [mss 1420,sackOK,TS val 30800
60133 ecr 0,nop,wscale 6], length 0
    0x0000: 4500 003c 5f9d 4000 4006 5ff6 ac11 0002  E...<_@.
    @_.....
    0x0010: 6cb1 6264 8118 0050 49d0 6612 0000 0000  1.bd...P
    I.f.....#
    0x0020: 8010 03fd 7b4f 0000 0101 080a b795 fce6  ....{O..
    .....
    0x0030: 9c40 e16a                                .@.j
08:45:17.369938 IP 172.17.0.2.33048 > 108.177.98.100.80: Flags [P.
], seq 1:86, ack 1, win 1021, options [nop,nop,TS val 3080060134 e
cr 2621497706], length 85: HTTP: GET / HTTP/1.1
    0x0000: 4500 0089 5f9f 4000 4006 5fa7 ac11 0002  E..._.@.
    @_.....
    0x0010: 6cb1 6264 8118 0050 49d0 6613 e682 cf23  1.bd...P
    I.f.....#
    0x0020: 8018 03fd 7ba4 0000 0101 080a b795 fce6  ....{...
    .....
    0x0030: 9c40 e16a 4745 5420 2f20 4854 5450 2f31  .@.jGET.
/.HTTP/1
    0x0040: 2e31 0d0a 486f 7374 3a20 6f70 656e 736f  .1..Host
:.openso
    0x0050: 7572 6365 2e67 6f6f 676c 652e 636f 6d0d  urce.goo

```

CONCLUSION

I have gained practical experience to enable you to

- identify network interfaces,
- use the `tcpdump` command to capture network data for inspection,
- interpret the information that `tcpdump` outputs regarding a packet, and
- save and load packet data for later analysis.

